CloudAge

● VIBE CODING GUIDE · 2026 EDITION

The 6 Documents You Need Before Writing Any Code

A complete framework for founders, indie hackers, and AI-era builders. Updated for Claude Code, Cursor, Windsurf, and modern stacks.

These six documents are the briefing your AI agent needs on Day 1. Write them once. Ship faster forever.

01 PRD
02 TRD
03 App Flow
04 UI/UX Brief
05 Backend Schema
06 Implementation Plan

BUILD FOR THE CLOUD ERA

INTRODUCTION

Why these documents matter

AI coding agents are powerful — but they are not mind readers. The more precise your context, the faster and more accurately they build.

These 6 documents eliminate ambiguity. Instead of the AI guessing your stack, navigation flow, or data model — it *knows*. The result: fewer hallucinations, less back-and-forth, cleaner architecture from day one.

Think of them as the briefing you'd give a senior developer on Day 1. Except this developer reads 10,000 tokens per second and forgets everything between sessions — so your documents are the memory it needs.

> **Agent context files:** Modern AI coding agents read a project memory file in your repo root. Claude Code reads `CLAUDE.md`. Cursor reads `.cursorrules`. Windsurf reads `.windsurfrules`. These 6 documents form the raw content for that file — write them once, paste them in, and your agent stays coherent across every session.

The 6 documents cover every axis your agent needs to make decisions: **what you're building** (PRD), **how you're building it** (TRD), **how users navigate it** (App Flow), **how it looks** (UI/UX), **how data is structured** (Backend Schema), and **in what order to build it** (Implementation Plan).

01

PRD — Product Requirements Doc

What you're building, for whom, and what success looks like

02

TRD — Technical Requirements Doc

Your stack, tools, APIs, and architectural constraints

03

App Flow

Every screen, every click, every navigation path — mapped

04

UI/UX Design Brief

Visual language: colors, typography, components, feel

05

Backend Schema

Tables, relationships, auth rules, and data structure

06

Implementation Plan

Phased build sequence so foundations always come first

DOCUMENT 01

PRD — Product Requirements Document

The north star. Everything else flows from this.

WHY IT MATTERS

The PRD is the first document you write — before any code, design, or architecture decisions. It defines what you're building, who it's for, and what "done" looks like. Without a PRD, your AI agent fills gaps with assumptions. Some will be right. Most won't be.

A solid PRD keeps every subsequent document honest. If a feature creeps into the TRD or schema that isn't in the PRD, that's a signal to stop and re-align.

WHAT TO INCLUDE

- App name and one-sentence tagline
- The problem being solved and who feels it
- Core value proposition — what makes this different
- Target user persona (2–3 sentences, specific)
- Must-have features for v1
- Nice-to-have features for v2+
- Explicit out-of-scope list
- User stories in "As a / I want / so that" format
- Success metrics — measurable, time-boxed
- AI/LLM integrations (if any) — what model, what task

TEMPLATE

FIELD	WHAT TO WRITE
App Name	e.g. `TaskFlow`
Tagline	One sentence. What does it do, for whom?
Problem	What pain does this solve? Who feels it acutely?
Target User	Describe your primary user in 2–3 specific sentences. Avoid generics like "anyone who wants to be productive."
Must-Have Features	List each on a new line. Be specific — not "auth" but "email + Google OAuth signup/login with magic link fallback."
Nice-to-Have	Features for v2 or if time allows. Keep this honest — if it's truly necessary, move it to Must-Have.
Out of Scope (v1)	Explicitly list what this version does NOT do. This prevents scope creep in both the agent and your own planning.
User Stories	*As a [user type], I want to [action] so that [outcome].* Write 3–5 for the core flows.
AI/LLM Features	e.g. "AI summarises meeting notes using Claude claude-sonnet-4-20250514. Input: transcript. Output: 5 bullet summary." Leave blank if not applicable.
Success Metrics	e.g. 200 signups in first month; core action completion rate >75%; churn <5% in month 1.

💡 **Agent tip:** Start every new Claude Code or Cursor session with: `Read my PRD and summarise your understanding of what we're building before touching any code.` This surfaces misunderstandings before they become bugs.

DOCUMENT 02

TRD — Technical Requirements Document

The blueprint that locks in your stack so the AI never guesses.

WHY IT MATTERS

The TRD translates your product vision into specific technical decisions. Without it, your AI agent picks a stack on its own — sometimes sensibly, often inconsistently across files. Worse, it may use deprecated patterns or libraries it saw frequently in training data.

Lock in your choices here. The agent will reference this document every time it makes a technical decision — imports, folder structure, auth patterns, and API design all flow from the TRD.

WHAT TO INCLUDE

- Frontend framework and version
- Backend runtime and framework
- Database type, provider, and ORM/client
- Authentication method and provider
- Hosting and deployment targets
- Third-party APIs and services
- Component/UI library
- Key utility libraries
- Environment variable names
- Runtime (Node, Bun, Deno)
- Hard constraints (mobile-only, free tier, etc.)
- Agent context

file location (CLAUDE.md / .cursorrules)

TEMPLATE — UPDATED FOR 2026

FIELD	WHAT TO WRITE
Runtime UPDATED	e.g. `Node.js 22 LTS` or `Bun 1.x` (recommended for speed). Specify one — don't let the agent mix them.
Frontend UPDATED	e.g. `Next.js 15 (App Router)` with TypeScript, Tailwind CSS v4. Specify App Router vs Pages Router explicitly.
Backend	e.g. `Next.js Server Actions` for full-stack; `Hono` on Cloudflare Workers for lightweight APIs; `FastAPI` for Python-heavy workloads.
Database UPDATED	e.g. `PostgreSQL via Neon` (serverless, generous free tier); `PostgreSQL via Supabase` ; `SQLite via Turso` for edge deployments.
ORM / DB Client NEW	e.g. `Drizzle ORM` (recommended — TypeScript-first, lightweight, Neon-compatible); `Prisma 6` ; `Kysely` for query builders.
Auth UPDATED	e.g. `Clerk` (recommended — drop-in, handles JWTs, orgs, MFA); `Supabase Auth` ; `NextAuth v5 (Auth.js)` ; `Lucia` for full control.
Hosting	e.g. `Vercel` (Next.js), `Cloudflare Pages` (edge), `Railway` or `Fly.io` (containers), `Render` (simple deploys).
UI Components NEW	e.g. `shadcn/ui` (recommended — copy-paste, Radix-based, Tailwind-native); `Radix UI primitives` ; `Mantine` .

FIELD	WHAT TO WRITE
Key Libraries UPDATED	e.g. `TanStack Query v5` (data fetching — formerly React Query); `Zod v3` (validation); `Lucide React` (icons); `date-fns`; `nuqs` (URL state).
Third-Party APIs	List each: name, purpose, free/paid tier, and which env var holds the key.
Environment Variables	List every variable name (not value). e.g. `DATABASE_URL`, `CLERK_SECRET_KEY`, `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`.
Constraints	e.g. Must work on mobile; must stay on free tier; no SSR (static export only); response time <500ms.

⚠️ **What changed:** `React Query` is now `TanStack Query v5` — the import path changed to `@tanstack/react-query`. Specifying this explicitly prevents the agent from using the old `react-query` package. Similarly, always pin your Next.js version (15 as of 2026) to avoid the agent referencing Pages Router patterns in an App Router project.

DOCUMENT 03

App Flow — Navigation & User Journey Map

Every page, every click, every path — mapped before a single screen is built.

WHY IT MATTERS

The app flow tells your AI agent exactly how users move through your product: what happens on button clicks, where users land after signup, and what the logged-out vs. logged-in experiences look like. Without it, the agent builds pages in isolation. They work individually but don't connect.

The app flow is what makes your product feel like one coherent thing instead of a collection of unrelated screens.

WHAT TO INCLUDE

- All screens/routes with short descriptions
- First screen for new visitors
- 2–3 core user journeys, step by step
- Error states and where users go
- All redirect logic
- Navigation structure (sidebar, tabs, navbar)
- Full auth flow sequence
- Empty states (no data yet)
- Modal, drawer, and overlay interactions
- AI streaming / loading states (if applicable)

TEMPLATE

FIELD	WHAT TO WRITE
All Routes	List every route: `/` (landing), `/login`, `/signup`, `/dashboard`, `/settings`, `/[id]`, etc. One line each with a one-sentence description.
Navigation Type	e.g. Top navbar (public) + left sidebar (app); bottom tab bar on mobile; breadcrumbs on nested pages.
First Screen	What does a brand-new visitor see? Is there a landing page or do they hit login directly?
Auth Flow	e.g. `/signup` → email verify → `/onboarding` (3 steps) → `/dashboard`. What happens if they skip onboarding?
Core Journey 1	Step-by-step: *User wants to [primary goal]. They go to... then click... then see... result is...*
Core Journey 2	Second most important flow. Follow same format.
Empty States	What renders when a list has no items yet? Add a CTA, illustration, or prompt — define it here so the agent builds it first time.
Error States	e.g. Failed payment → `/billing?error=true`; network error → inline toast; 404 → custom error page with home link.
AI Loading States NEW	If any screen calls an AI/LLM: does it stream output, show a skeleton, or block with a spinner? Define the pattern once here.
Redirects	e.g. After login → `/dashboard`; after logout → `/`; unauthenticated access to

FIELD	WHAT TO WRITE
	`/dashboard` → `/login?redirect=/dashboard`.

💡 **Agent tip:** Write your app flow as a numbered list of routes first, then describe each core journey as a short story. Agents parse narrative instructions well — "*the user clicks Save, sees a success toast for 3 seconds, and the list refreshes*" is more useful than "*implement save functionality.*"

DOCUMENT 04

UI/UX Design Brief — Visual & Interaction Guide

So the AI builds something you'd actually want to use.

WHY IT MATTERS

AI agents default to plain, inconsistent UIs when left to their own judgment. The design brief locks in your visual language: colors, typography, component style, and overall feel. Every screen the agent generates will draw from this.

You don't need to be a designer. Even a direction like "dark mode, minimal, like Linear" gives the agent enough to make coherent decisions. Vagueness is what produces generic output.

WHAT TO INCLUDE

- Overall aesthetic direction (one clear word or phrase)
- Typography choices (heading + body + mono)
- Border radius and shadow rules
- Reference apps (2–3 max)
- Mobile responsiveness requirements
- Full color palette with hex values
- Component library specification
- Dark / light mode preference
- Key UI patterns (cards, tables, sidebars)
- Animation / transition philosophy

TEMPLATE — UPDATED FOR 2026

FIELD	WHAT TO WRITE
Aesthetic	One clear direction. e.g. *Minimal, dark-mode first, data-dense — like Linear or Vercel dashboard.*
Primary Color	e.g. `#0052CC` (indigo-blue). Include a name so the agent references it consistently.
Background	e.g. `#0D0D0D` (dark) or `#FAFAFA` (light). One value. Commit.
Text Colors	e.g. Primary: `#F5F5F7` / Secondary: `#8A9DB5` / Muted: `#4A6580`
Accent / CTA	e.g. `#0066FF`. Used for buttons, links, focus rings. One color only.
Font — Heading	e.g. `Sora`, `Cal Sans`, `DM Serif Display`. Choose something with character.
Font — Body	e.g. `DM Sans`, `Geist`, `Plus Jakarta Sans`. Optimised for reading.
Font — Mono	e.g. `JetBrains Mono`, `Geist Mono`. For code blocks and labels.
UI Components UPDATED	e.g. `shadcn/ui` as the base component library. Specify which components to customise (Button, Card, Dialog, etc.).
Border Radius	e.g. `8px` rounded consistently; `4px` for inputs; `50%` for avatars only.
Shadows	e.g. Subtle card shadows only — `0 1px 3px rgba(0,0,0,0.12)`. No heavy drop shadows. Or: no shadows, borders only.
Animations NEW	e.g. Transitions: `150ms ease`. Page transitions: fade only. No bounce/

FIELD	WHAT TO WRITE
	spring except on toasts. Or: use Framer Motion with `spring` physics.
Dark / Light Mode	e.g. Dark mode primary, light mode optional. Specify CSS variable naming if using a token system.
Reference Apps	2–3 max. e.g. Linear (density + shortcuts), Vercel (dark minimal), Notion (content hierarchy).
Mobile	e.g. Fully responsive; sidebar collapses to hamburger at 768px; bottom tab nav on mobile; tap targets minimum 44px.

💡 **shadcn/ui note:** shadcn/ui is not a component library you install — it's a collection of copy-pasteable components built on Radix UI and Tailwind. Tell your agent: "*Use shadcn/ui. Run `npx shadcn@latest add button card dialog` for each component rather than installing a package.*" This prevents the agent from reaching for @mui or @chakra-ui by habit.

DOCUMENT 05

Backend Schema — Data Model & Auth

Architecture

How your data is stored, structured, and secured — defined before any migrations are written.

WHY IT MATTERS

The backend schema is the hardest document to change after launch. Once your database has real data, restructuring tables is painful — it means migrations, backfilling, and potential downtime. Getting this right upfront saves days of debugging.

Your agent uses this to write database migrations, RLS policies, API routes, and auth logic. The more precise you are, the fewer security holes end up in your codebase.

WHAT TO INCLUDE

- All database tables with columns and types
- Indexes (which columns need fast lookup)
- User roles and permissions
- File/media storage structure
- API endpoint list (optional but helpful)
- Primary keys and foreign key relationships
- Auth model and Row Level Security rules
- Sensitive fields (what stays out of the DB)
- ORM schema format (Drizzle / Prisma)
- Vector embeddings table (if using AI search)

TEMPLATE — UPDATED FOR 2026

FIELD	WHAT TO WRITE
DB Provider UPDATED	e.g. `Neon` (serverless Postgres, recommended — auto-suspend saves cost); `Supabase Postgres`; `Turso` (SQLite at the edge). Pick one.
ORM Format NEW	Specify your ORM so the agent writes schema in the right format. e.g. "*Write all schemas as Drizzle ORM TypeScript definitions.*" or "*Use Prisma schema format.*"
Table: users	`id` (uuid, PK), `email` (text, unique), `name` (text), `role` (text, default 'user'), `created_at` (timestamp, default now())
Table: [your table]	`id` (uuid, PK), `user_id` (uuid, FK → users.id, cascade delete), [your columns], `created_at`, `updated_at`
Add tables…	Define every table your app needs. One row per table. Better to over-specify here than under-specify.
Relationships	e.g. `projects.user_id → users.id` (many-to-one); `tasks.project_id → projects.id` (many-to-one, cascade delete).
Indexes	e.g. Index on `tasks.user_id`, `tasks.status`, `tasks.created_at DESC` for the main list view.
Auth Provider	e.g. `Clerk` — JWTs handled externally; store `clerk_user_id` in users table as the foreign key bridge. Or `Supabase Auth` — uses built-in `auth.users` table.
Row Level Security	e.g. `users can only SELECT/INSERT/UPDATE/DELETE their own rows`. Write one RLS rule per table. Be explicit —

FIELD	WHAT TO WRITE
	agents default to permissive if not told otherwise.
User Roles	e.g. `admin`: full access. `user`: own data only. `guest`: read-only public data.
File Storage	e.g. Supabase Storage — `/avatars/{user_id}`, `/uploads/{user_id}/{file_id}`. Or Cloudflare R2 for larger volumes.
Sensitive Fields	e.g. `payment_method` — stored via Stripe, never in your DB. `api_keys` — store hashed only, never plaintext.
Vector Embeddings NEW	If using AI semantic search: add a `embeddings` table with `vector(1536)` column (pgvector). Or use Supabase's built-in vector support. Specify here if needed, skip if not.

💡 **Drizzle vs. Prisma:** Both are solid choices. Drizzle is TypeScript-first, generates less boilerplate, and works natively with Neon/Turso serverless drivers. Prisma has a larger ecosystem and is more familiar to many agents from training data. Specify your choice explicitly — an agent defaulting to Prisma in a Drizzle project will generate unusable code.

DOCUMENT 06

Implementation Plan — Phased

Build Sequence

The exact order to build so foundations always come before features.

WHY IT MATTERS

The implementation plan is your build roadmap. It tells the agent what to build first, second, and third — and why that order matters. Without it, agents routinely build UI before the database exists, or wire up auth after the schema is half-written.

Think of it as a checklist the agent ticks off. Clear phases, clear milestones, clear "done" criteria. The more specific you are, the less the agent freelances.

WHAT TO INCLUDE

- Numbered phases with explicit goals
- Phase 0: Agent context file and conventions
- Phase 1: Project setup, repo, env vars
- Phase 2: Database schema and migrations
- Phase 3: Authentication flow
- Phase 4+: Core features in dependency order
- UI polish phase (always after core features)
- Testing and edge case phase
- Deployment and production config phase
- Explicit "done" criteria per phase

TEMPLATE — UPDATED FOR 2026

PHASE	GOAL & TASKS
Phase 0 NEW **Agent Context**	Create `CLAUDE.md` / `.cursorrules` in the repo root. Paste the condensed TRD, naming conventions, folder structure, and DO-NOT rules. This file persists across every agent session. Done: agent can answer "what stack are we using?" correctly from context alone.
Phase 1 Setup	Initialise project (`bun create next-app` or equivalent), install all dependencies from TRD, configure TypeScript paths, set up Tailwind, create `.env.local` with all variable names (values empty). Push to repo. Done: `dev` server runs with no errors.
Phase 2 Database	Write all table schemas in Drizzle/Prisma, run initial migration, configure RLS policies, seed test data. Done: every table exists, RLS is active, seed data is queryable.
Phase 3 Auth	Integrate Clerk/Supabase Auth/NextAuth. Implement signup, login, logout, session handling, and protected route middleware. Sync auth user to your `users` table. Done: a new user can sign up, log in, see `/dashboard`, and log out cleanly.
Phase 4 Core Feature 1	Describe the feature and its sub-tasks in order. Build API/server action first, then the UI that calls it. Done: the primary user journey from App Flow doc works end-to-end with real data.
Phase 5 Core Feature 2	Next most important feature. Same structure — data layer first, then UI. Done: second core journey works.
Continue…	

PHASE	GOAL & TASKS
	Add a phase for each core feature. Never skip ahead to UI polish while a core feature is incomplete.
Phase N **UI Polish**	Responsive design on all breakpoints, loading skeletons, empty states, error boundaries, toast notifications. Reference the UI/UX design brief for each decision. Done: app looks intentional, not like a default template.
Phase N+1 **Testing**	Manually test all user journeys from App Flow doc. Fix edge cases. Add error handling for all async operations. Test on mobile viewport. Done: all core flows work without errors on desktop and mobile.
Phase N+2 **Deploy**	Set up production environment, configure all env vars in Vercel/Railway/ etc., point custom domain, configure CORS, enable rate limiting. Done: app is live at production URL, all flows work in prod.
Done Criteria	Define "finished" for the whole project. e.g. *All core user journeys from the App Flow doc complete successfully; no console errors in prod; app loads in under 2s on mobile.*

💡 **The rule of Phase 0:** Before any code exists, the agent context file (`CLAUDE.md` or `.cursorrules`) should exist. This is the single biggest unlock for multi-session consistency. Every time you start a new Claude Code session, it reads this file first. Without it, you're re-briefing the agent from scratch every time.

WORKFLOW

How to use these documents

You don't write all six from scratch. You fill in templates, let an AI help clean them up, and ship them into your agent's context before the first line of code.

The suggested workflow below takes 30–60 minutes the first time. It saves hours of fixing broken architecture, re-explaining context mid-build, and undoing bad decisions made by an agent that didn't have enough information.

1. **Start with the PRD — get the idea out of your head**

   Open a conversation with Claude and say: "*I want to build [your idea]. Help me fill in this PRD template.*" Answer its questions conversationally. Let it clean up the prose.

2. **Fill the TRD — decide your stack before writing anything**

   Don't skip this step. Pick your framework, database, auth provider, and ORM now. The agent will reference this in every file it writes. Changing the stack mid-build costs hours.

3. **Map your App Flow — draw it first, describe it second**

   Sketch the screens on paper or in Excalidraw. Then translate that sketch into a text description. The act of drawing surfaces edge cases you'd otherwise discover in code.

4. **Write the UI/UX brief — even a rough direction helps**

   If you have no design opinion: pick two reference apps you like, specify dark or light, and name a font. That's enough. If you have a clear vision, be specific with hex values and reference screenshots.

5

Design the Backend Schema — tables first, relationships second

List every noun in your product (user, project, task, message, subscription). Each noun is probably a table. Define columns, then draw the lines between them. Your agent will generate migrations directly from this.

6

Write the Implementation Plan — phases, not a flat task list

A flat list of 50 tasks fails. Phases with goals succeed. Each phase should produce something testable and shippable before the next phase starts.

7

Create your agent context file (`CLAUDE.md` / `.cursorrules`)

Condense the TRD, folder structure, naming conventions, and critical constraints into a single file at the repo root. This is the persistent memory your agent reads at the start of every session.

WHICH AGENT TO USE

Claude Code

Anthropic's CLI coding agent. Works directly in your terminal with full repo access. Reads `CLAUDE.md` automatically on startup.

```
Context file: CLAUDE.md
```

Cursor

AI-first IDE (VS Code fork). Inline code generation + Composer for multi-file edits. Reads `.cursorrules` as project context.

```
Context file: .cursorrules
```

Windsurf

AI IDE by Codeium. Similar to Cursor with its own "Cascade" agent mode for multi-step tasks. Reads `.windsurfrules`.

Bolt / Lovable / v0

Full-stack browser-based agents. Great for rapid prototyping. Paste your 6 documents in the first message as system context.

```
Context: paste in first message
```

```
Context
file: .windsurfrules
```

STARTER PROMPT

PASTE THIS AT THE START OF YOUR FIRST AGENT SESSION

```
I'm building a new project. Here are my 6 planning
documents.
Use these as the single source of truth for all decisions.

Do not use any libraries, frameworks, or patterns that
contradict the TRD. Do not build features outside the PRD
scope.
Before you write any code, confirm your understanding of:
1. The stack we're using
2. The first screen a new user sees
3. The primary user journey

[PASTE YOUR 6 DOCUMENTS BELOW]
```

Realistic time estimate: Completing all 6 documents takes 45–90 minutes for a focused builder who knows their product idea. That investment eliminates an estimated 8–20 hours of rebuilding, redirecting, and debugging caused by an under-informed agent. It's the highest-leverage hour you'll spend before launch.